

comparator 정리

정렬이나 우선순위 큐에서 두 값을 어떤 기준으로 비교할지 정하는 규칙입니다.

사용처 (Where)

- 정렬 기준을 직접 정할 때
- 구조체를 특정 필드로 정렬할 때
- priority_queue의 우선순위를 바꿀 때

방법 (How to Use)

- sort에는 bool을 반환하는 함수/lambda 전달
- a가 b보다 먼저 와야 하면 true
- priority_queue는 비교 방향이 헷갈리므로 예제로 확인
- 같은 값 처리 기준을 명확히 작성

기본 정보

- 필요 헤더: #include <algorithm>, #include <queue>
- 기본 선언: sort(v.begin(), v.end(), comp);

왜 써야 할까? (Why)

- 데이터 정렬 기준을 코드로 명확하게 표현
- 복잡한 구조체도 원하는 기준으로 처리 가능
- priority_queue 응용에 필수

주요 함수

함수/문법	의미
lambda comparator	짧은 기준
function comparator	재사용 기준
struct comparator	priority_queue에 자주 사용
greater<T>	기본 제공 내림/오름 보조

기본 코드 예시

```
#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

int main() {
    vector<pair<int, string>> v = {{90, "Kim"}, {90, "Lee"}, {80, "Park"}};
    sort(v.begin(), v.end(), [](auto a, auto b) {
        if (a.first != b.first) return a.first > b.first;
        return a.second < b.second;
    });
}
```

comparator 비교와 응용

STL을 직접 구현이나 C 스타일 코드와 비교하고, 실제 문제에서 쓰는 방식을 확인합니다.

시간 복잡도

동작	복잡도
sort 비교	$O(n \log n)$ 번 호출
priority_queue push/pop	$O(\log n)$
비교 자체	보통 $O(1)$

STL vs 직접 구현 vs C 스타일

구분	STL	직접 구현	C 스타일
표현력	의도를 타입으로 직접 표현	구조체/클래스를 직접 작성	배열/struct 수동 관리
코드 길이	짧고 표준적인 형태	필드와 생성자 작성 필요	초기화와 접근 실수 가능
유지보수	다른 STL과 자연스럽게 연동	설계가 늘어날수록 관리 필요	가독성이 떨어지기 쉬움
추천 상황	간단한 값 묶음/조건 전달	도메인 의미가 큰 객체	C API와 맞춰야 할 때

응용: 우선순위 큐에서 작은 점수 우선

struct 비교자를 만들어 priority_queue가 낮은 점수를 먼저 꺼내게 합니다.

```
#include <iostream>
#include <queue>
#include <vector>
using namespace std;

struct Node {
    int score;
    string name;
};

struct Compare {
    bool operator()(const Node& a, const Node& b) {
        return a.score > b.score;
    }
};

int main() {
    priority_queue<Node, vector<Node>, Compare> pq;
    pq.push({90, "Kim"});
    pq.push({80, "Lee"});

    cout << pq.top().name;
}
```

처음 배울 때 자주 하는 실수

- sort 비교 함수에 <=를 쓰지 않기
- 동점 처리 규칙을 빠뜨리면 결과가 불안정할 수 있음
- priority_queue 비교 방향은 sort와 다르게 느껴질 수 있어 테스트 필요

비슷한 항목과 차이

- lambda는 짧은 comparator 작성에 좋음
- greater<T>는 기본 타입에 간단히 사용
- stable_sort는 동점의 기존 순서를 보존

한 줄 정리: comparator은(는) '정렬 기준을 직접 정할 때' 상황에서 먼저 떠올리면 좋은 STL입니다.